Generative Adversarial Networks (Practicals)

Structured Notes

Vanilla GANs, Deep Convolutional GANs (DCGANs), Adversarial Training, Image Generation

Contents

1 Generative Adversarial Networks — Overview **2**
1.1 Why “Adversarial”? . . . 2
1.2 Applications of GANs . . . 2

2 Vanilla GAN **2**
2.1 Data Preparation . . . 2
2.2 Latent Space Vector . . . 3
2.3 Reusable Block Pattern . . . 3
2.4 Generator Architecture . . . 4
2.5 Discriminator Architecture . . . 4
2.6 Training the Vanilla GAN . . . 5
2.6.1 Loss Function and Optimizers . . . 5
2.6.2 Training Loop . . . 6

3 Deep Convolutional GAN (DCGAN) **7**
3.1 Why DCGANs are Better than Vanilla GANs . . . 7
3.2 Key Architectural Differences . . . 7
3.3 Transposed Convolution vs. Convolution . . . 7
3.4 DCGAN Applications . . . 8
3.5 DCGAN Generator Architecture . . . 8
3.6 DCGAN Discriminator Architecture . . . 9
3.7 DCGAN Training . . . 10

4 Key Concepts **10**
4.1 Batch Normalization . . . 10
4.2 ReLU vs. Leaky ReLU . . . 11
4.3 Checkpoints vs. Models . . . 11

5 Summary **11**

1 Generative Adversarial Networks — Overview

Generative Adversarial Network (GAN)

A **Generative Adversarial Network (GAN)** is a deep learning framework that internally consists of two neural network architectures:

- **Generator**: A neural network that creates synthetic data from random noise
- **Discriminator**: A neural network that distinguishes whether data is real or fake

Both networks are trained **adversarially** — the generator learns to produce data that closely resembles real-world data, while the discriminator learns to correctly identify fake data.

1.1 Why "Adversarial"?

The framework is called **adversarial** because the two networks **compete against one another**:

- The **generator** tries its best to **fool the discriminator** by generating data that appears real
- The **discriminator** tries its best to **correctly identify** whether the data the generator produced is real or fake

This competitive dynamic drives both networks to improve continuously during training.

1.2 Applications of GANs

- **Image generation**: Generating a large number of realistic images
- **Image super-resolution**: Generating high-quality images from low-quality images
- **Image-to-image translation**: Converting one type of image into another
- **Data generation**: Generating large amounts of training data for neural networks
- **Medical imaging**: Enhancing or generating data for medical image analysis
- **Autonomous vehicles**: Generating road images in various conditions (cloudy, sunny, foggy, night-time) where real-life data is limited; enhancing low-quality camera images captured in poor conditions

2 Vanilla GAN

Vanilla GAN

A **Vanilla GAN** is the basic form of Generative Adversarial Networks. It uses **simple linear (fully connected) layers** in both the generator and discriminator parts of the network. Vanilla GANs are suitable for simple tasks such as generating images from the MNIST dataset.

Simple vs. Complex Tasks

Vanilla GANs work well for generating simple images (e.g., MNIST digit images of size 28×28). However, for complex tasks such as super-resolution, medical imaging, or segmentation, Vanilla GANs are not suitable — more advanced architectures like DCGANs are needed.

2.1 Data Preparation

Data Loading and Preprocessing

```python
import torch
import torchvision
from torchvision import transforms
from torch.utils.data import DataLoader

# Image size for MNIST
```

```
mnist_image_size = 28

# Batch size
batch_size = 128

# Transform: convert to tensor and normalize to [-1, 1]
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Load MNIST dataset
train_dataset = torchvision.datasets.MNIST(
    root='./data', train=True, download=True,
    transform=transform)

train_loader = DataLoader(train_dataset,
    batch_size=batch_size, shuffle=True)
# Training: 60,000 images, Testing: 10,000 images
```

Listing 1: Loading MNIST Dataset for Vanilla GAN

Normalization to $[-1, 1$

$]The transform normalizes pixel values to the range [-1, 1] instead of [0, 255]. This is essential because neural networ$

2.2 Latent Space Vector

A random vector of size 100 is created as input for the generator. This vector contains 100 randomly generated values from a Gaussian distribution.

Latent Vector Generation

```
latent_space_dim = 100

def get_gaussian_latent_vectors(batch_size):
    return torch.randn(batch_size, latent_space_dim)
```

Listing 2: Generating Random Noise Vectors

Each latent vector has 100 values, and for each batch, 128 such vectors are generated (batch size of 128 vectors, each of dimension 100).

2.3 Reusable Block Pattern

Both the generator and discriminator are built using a reusable **vanilla_block** helper function. This pattern keeps the architecture concise:

Vanilla Block Helper

```
def vanilla_block(in_feat, out_feat, normalize=True,
                  activation=None):
    layers = [nn.Linear(in_feat, out_feat)]
    if normalize:
        layers.append(nn.BatchNorm1d(out_feat))
    layers.append(nn.LeakyReLU(0.2)
                  if activation is None else activation)
    return layers
```

Listing 3: Reusable Block for Vanilla GAN

The `normalize` parameter controls whether Batch Normalization is applied. The generator uses normalization, but the discriminator does not.

2.4 Generator Architecture

The generator consists of **four blocks**, each using the vanilla_block with normalization enabled (except the last block which uses Tanh):

Block	Input	Output	Layers
Block 1	100	256	Linear + BatchNorm + LeakyReLU
Block 2	256	512	Linear + BatchNorm + LeakyReLU
Block 3	512	1024	Linear + BatchNorm + LeakyReLU
Block 4	1024	784	Linear + Tanh (no BatchNorm)

The final output size is **784** (28×28), which is the flattened size of an MNIST image. The output is reshaped into a 28×28 image.

Vanilla GAN Generator

```
class GeneratorNet(nn.Module):
    def __init__(self, img_shape=(28, 28)):
        super().__init__()
        self.generated_img_shape = img_shape
        num_neurons = [LATENT_SPACE_DIM, 256, 512, 1024,
                       img_shape[0] * img_shape[1]]

        self.net = nn.Sequential(
            *vanilla_block(num_neurons[0], num_neurons[1]),
            *vanilla_block(num_neurons[1], num_neurons[2]),
            *vanilla_block(num_neurons[2], num_neurons[3]),
            *vanilla_block(num_neurons[3], num_neurons[4],
                           normalize=False,
                           activation=nn.Tanh())
        )

    def forward(self, latent_vector_batch):
        img_flat = self.net(latent_vector_batch)
        return img_flat.view(img_flat.shape[0], 1,
                             *self.generated_img_shape)
```

Listing 4: Generator Network with Linear Layers

2.5 Discriminator Architecture

The discriminator consists of **three blocks**. Unlike the generator, the discriminator does **not use Batch Normalization** (`normalize=False`):

Block	Input	Output	Layers
Block 1	784	512	Linear + LeakyReLU (no BatchNorm)
Block 2	512	256	Linear + LeakyReLU (no BatchNorm)
Block 3	256	1	Linear + Sigmoid (no BatchNorm)

The input to the discriminator is a flattened image (784 values), and the output is a single value indicating real (close to 1) or fake (close to 0). The discriminator outputs a probability — 1.0 means 100% sure the image is real, 0.5 means 50% sure.

Discriminator Does Not Use BatchNorm

Using Batch Normalization in the Vanilla GAN discriminator does not work well. The discriminator only uses Linear layers followed by LeakyReLU activation, without normalization layers.

Vanilla GAN Discriminator

```python
class DiscriminatorNet(nn.Module):
    def __init__(self, img_shape=(28, 28)):
        super().__init__()
        num_neurons = [img_shape[0] * img_shape[1],
                       512, 256, 1]

        # normalize=False for ALL discriminator blocks
        self.net = nn.Sequential(
            *vanilla_block(num_neurons[0], num_neurons[1],
                           normalize=False),
            *vanilla_block(num_neurons[1], num_neurons[2],
                           normalize=False),
            *vanilla_block(num_neurons[2], num_neurons[3],
                           normalize=False,
                           activation=nn.Sigmoid())
        )

    def forward(self, img_batch):
        img_flat = img_batch.view(img_batch.shape[0], -1)
        return self.net(img_flat)
```

Listing 5: Discriminator Network — No Batch Normalization

2.6 Training the Vanilla GAN

2.6.1 Loss Function and Optimizers

Training Setup

```python
# Adam optimizer for both networks
optimizer_G = torch.optim.Adam(generator.parameters(),
    lr=0.0002, betas=(0.5, 0.999))
optimizer_D = torch.optim.Adam(discriminator.parameters(),
    lr=0.0002, betas=(0.5, 0.999))

# Binary Cross Entropy loss (adversarial loss)
adversarial_loss = nn.BCELoss()

# Ground truth labels
real_images_gt = torch.ones(batch_size, 1)    # Tensor of 1s
fake_images_gt = torch.zeros(batch_size, 1)   # Tensor of 0s
```

Listing 6: Optimizers and Loss Function

Real = 1, Fake = 0

In adversarial networks:

- **Real** data is represented by a tensor of **ones** (1)
- **Fake** data is represented by a tensor of **zeros** (0)

The adversarial loss (BCE) measures how much the discriminator's predictions deviate from these

ground truth labels.

2.6.2 Training Loop

Training order (critical):

1. **Generator generates** fake images from random noise
2. **Discriminator is updated first:**
   - Computes loss on fake images: BCE(D(fake), **0**)
   - Computes loss on real images: BCE(D(real), **1**)
   - Combined discriminator loss updates discriminator weights
3. **Generator is updated second:**
   - Uses discriminator's predictions on generated images
   - Computes loss: BCE(D(fake), **1**) — generator wants discriminator to predict its output as **real**
   - Generator loss updates generator weights

Why Discriminator Updates First

The discriminator must be updated before the generator because the generator loss depends on the discriminator's output. The discriminator's predictions on generated images are used to compute the generator's loss. If the generator were updated first, the loss computation would be based on an outdated discriminator.

Training Loop

```
num_epochs = 10

for epoch in range(num_epochs):
    for batch_idx, (real_images, _) in enumerate(
            mnist_data_loader):
        real_images = real_images.to(device)

        # --- Train Discriminator ---
        discriminator_opt.zero_grad()

        # -log(D(x)): D should output 1 for real
        real_d_loss = adversarial_loss(
            discriminator_net(real_images), real_images_gt)

        # G(z): generate fake images
        fake_images = generator_net(
            get_gaussian_latent_batch(batch_size, device))
        # D(G(z)): detach() prevents G gradients
        fake_preds = discriminator_net(
            fake_images.detach())
        # -log(1-D(G(z))): D should output 0 for fake
        fake_d_loss = adversarial_loss(
            fake_preds, fake_images_gt)

        d_loss = real_d_loss + fake_d_loss
        d_loss.backward()
        discriminator_opt.step()

        # --- Train Generator ---
        generator_opt.zero_grad()

```

```
        # D(G(z)): generate new fakes and evaluate
        gen_preds = discriminator_net(generator_net(
            get_gaussian_latent_batch(batch_size, device)))
        # Use real_images_gt (ones) to fool D
        g_loss = adversarial_loss(gen_preds,
                                  real_images_gt)

        g_loss.backward()
        generator_opt.step()
```

Listing 7: Adversarial Training Loop

Generator Loss Uses Real Labels

When computing the generator loss, the ground truth is `real_images_gt` (tensor of ones), not fake labels. This is because the generator's goal is to produce images that the discriminator classifies as **real**. The loss measures how far the discriminator's output is from 1 (real) when evaluating the generator's output.

3 Deep Convolutional GAN (DCGAN)

DCGAN

A **Deep Convolutional GAN (DCGAN)** is an improved type of Generative Adversarial Network that uses **convolutional neural networks** in place of fully connected layers to generate realistic data. The generator uses **transposed convolution** layers and the discriminator uses standard **convolution** layers.

3.1 Why DCGANs are Better than Vanilla GANs

- Vanilla GANs use only fully connected layers, which are **not able to capture spatial relationships** in images (edges, textures, patterns)
- DCGANs use convolutional layers, which **capture both simple features and complex patterns**
- DCGANs produce **better quality** outputs compared to Vanilla GANs

3.2 Key Architectural Differences

Component	Vanilla GAN	DCGAN
Generator layers	Fully connected (linear)	Transposed convolution
Discriminator layers	Fully connected (linear)	Convolution
Spatial features	Cannot capture well	Captures edges, textures, patterns
Suitable tasks	Simple images (MNIST)	Complex tasks (super-resolution, medical)

3.3 Transposed Convolution vs. Convolution

Transposed Convolution vs. Convolution

- **Convolution**: Extracts features from the image while **decreasing** the spatial dimension (down-sampling). Used in the **discriminator** (encoder parts).
- **Transposed Convolution**: **Increases** the spatial dimension by upsampling. Used in the **generator** (decoder parts).

Convolution is used in encoder parts and transposed convolution is used in decoder parts of neural networks.

3.4 DCGAN Applications

- Image super-resolution
- Image generation
- Medical imaging
- Image-to-image translation
- Data generation for training (e.g., generating road images for autonomous vehicle training in various conditions)

3.5 DCGAN Generator Architecture

The generator takes a latent vector and produces a 32×32 image through a projection layer followed by transposed convolution operations:

1. **Project layer**: Linear converts the latent vector (size 100) to $4 \times 4 \times 256$ feature map, followed by BatchNorm1d and ReLU
2. **Reshape**: The flat vector is reshaped to a $(256, 4, 4)$ feature map
3. **Transpose Convolution blocks**: Four ConvTranspose2d layers progressively upsample: $4 \to 8 \to 16 \to 32$
4. **Final layer**: ConvTranspose2d with Tanh producing a single-channel 32×32 image

Layer	Channels	Spatial Size	Details
Project (Linear)	256	4×4	BatchNorm1d + ReLU
ConvTranspose2d	$256 \to 64$	$4 \to 8$	kernel=5, stride=2, pad=2, out_pad=1
ConvTranspose2d	$64 \to 32$	$8 \to 16$	kernel=5, stride=2, pad=2, out_pad=1
ConvTranspose2d	$32 \to 16$	$16 \to 32$	kernel=5, stride=2, pad=2, out_pad=1
ConvTranspose2d	$16 \to 1$	32×32	kernel=5, stride=1, pad=2 + Tanh

Generator Outputs 32×32 Images

The DCGAN generator produces 32×32 images (not 28×28). Because the transposed convolutions with stride=2 double the spatial dimensions ($4 \to 8 \to 16 \to 32$), the output is 32×32. During training, real MNIST images (28×28) are **upsampled to** 32×32 using `nn.Upsample(size=(32, 32))` to match the generator's output size.

DCGAN Generator

```
class GeneratorNet(nn.Module):
    def __init__(self, latent_dim=LATENT_SPACE_DIM):
        super().__init__()
        # Project: latent vector -> 4x4x256 feature map
        self.project = nn.Sequential(
            nn.Linear(latent_dim, 4 * 4 * 256),
            nn.BatchNorm1d(4 * 4 * 256),
            nn.ReLU(True)
        )
        # Transposed convolution layers: 4->8->16->32
        self.net = nn.Sequential(
            nn.ConvTranspose2d(256, 64, kernel_size=5,
                stride=2, padding=2, output_padding=1),
            nn.BatchNorm2d(64), nn.ReLU(True),
            nn.ConvTranspose2d(64, 32, kernel_size=5,
```

```
                stride=2, padding=2, output_padding=1),
            nn.BatchNorm2d(32), nn.ReLU(True),
            nn.ConvTranspose2d(32, 16, kernel_size=5,
                stride=2, padding=2, output_padding=1),
            nn.BatchNorm2d(16), nn.ReLU(True),
            nn.ConvTranspose2d(16, 1, kernel_size=5,
                stride=1, padding=2),
            nn.Tanh()
        )

    def forward(self, z):
        x = self.project(z)
        x = x.view(-1, 256, 4, 4)  # Reshape to feature map
        return self.net(x)  # Output: (N, 1, 32, 32)
```

Listing 8: DCGAN Generator with Transposed Convolutions

3.6 DCGAN Discriminator Architecture

The discriminator uses standard convolution layers with `kernel_size=4`, `stride=2`, `padding=1`. The first convolution layer does **not** use Batch Normalization, but layers 2 and 3 do:

Layer	In Ch.	Out Ch.	Spatial	Activation
Conv Layer 1	1	16	$32 \to 16$	LeakyReLU (no BN)
Conv Layer 2	16	32	$16 \to 8$	BatchNorm + LeakyReLU
Conv Layer 3	32	64	$8 \to 4$	BatchNorm + LeakyReLU
Classifier	$64 \times 4 \times 4$	1	—	Flatten + Linear + Sigmoid

The classifier flattens the $64 \times 4 \times 4 = 1024$ features into a single output value via `nn.Linear(64 * 4 * 4, 1)`.

DCGAN Discriminator

```
class DiscriminatorNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            # Layer 1: 1->16, no BatchNorm
            nn.Conv2d(1, 16, kernel_size=4,
                      stride=2, padding=1),
            nn.LeakyReLU(0.2, inplace=True),
            # Layer 2: 16->32, with BatchNorm
            nn.Conv2d(16, 32, kernel_size=4,
                      stride=2, padding=1),
            nn.BatchNorm2d(32),
            nn.LeakyReLU(0.2, inplace=True),
            # Layer 3: 32->64, with BatchNorm
            nn.Conv2d(32, 64, kernel_size=4,
                      stride=2, padding=1),
            nn.BatchNorm2d(64),
            nn.LeakyReLU(0.2, inplace=True),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 4 * 4, 1),  # 32->16->8->4
            nn.Sigmoid()
        )

```

```
    def forward(self, img):
        x = self.net(img)
        return self.classifier(x)
```

Listing 9: DCGAN Discriminator with Convolution Layers

3.7 DCGAN Training

The training process follows the same adversarial structure as Vanilla GAN, with one critical difference: **real images must be upsampled to** 32×32 to match the generator's output size.

DCGAN Training Setup

```
# Same optimizer configuration
optimizer_G = Adam(generator_net.parameters(),
    lr=0.0002, betas=(0.5, 0.999))
optimizer_D = Adam(discriminator_net.parameters(),
    lr=0.0002, betas=(0.5, 0.999))

adversarial_loss = nn.BCELoss()
num_epochs = 10

for epoch in range(num_epochs):
    for batch_idx, (real_images, _) in enumerate(loader):
        real_images = real_images.to(device)

        # CRITICAL: Resize real images to 32x32
        # to match generator output size
        real_images_resized = nn.Upsample(
            size=(32, 32), mode='nearest')(real_images)

        # Train D on resized real images
        real_loss = adversarial_loss(
            discriminator_net(real_images_resized),
            real_images_gt)
        # ... rest of training is same as Vanilla GAN
```

Listing 10: DCGAN Training with Image Upsampling

Learning Rate for GANs

The learning rate for GANs is generally kept in the range around 0.0002. If the learning rate is too high, training becomes **unstable**. If it is too low, training becomes very **slow**. This range provides a good balance for Generative Adversarial Networks.

4 Key Concepts

4.1 Batch Normalization

Batch Normalization

Batch normalization normalizes the output of each layer by computing the **mean** and **standard deviation** of the entire batch, then scaling and shifting the output so that it falls within a uniform range. It is equivalent to applying normalization at the output of each layer.

How batch normalization works:

1. Compute the mean and variance of the entire batch

2. Normalize each output by subtracting the mean and dividing by the standard deviation
3. Apply learned scale (gamma) and shift (beta) parameters

This ensures the output of each layer is in a normalized range, which stabilizes training.

4.2 ReLU vs. Leaky ReLU

Property	ReLU	Leaky ReLU
Negative values	Output is 0	Outputs a small value
Vanishing gradient	Can cause dead neurons	Prevents dead neurons
Output range	$[0, \infty)$	$(-\infty, \infty)$
Used in GANs	Generator (sometimes)	Discriminator (preferred)

Dead Neurons

With standard ReLU, neurons that output zero have a gradient of zero. These neurons will never be updated during training and are called **dead neurons**. Leaky ReLU prevents this by providing a small non-zero gradient for negative inputs, ensuring all neurons participate in the output generation.

4.3 Checkpoints vs. Models

Aspect	Checkpoint	Model
Definition	Intermediate training state saved periodically	Complete neural network with all parameters
Content	Saves only weights at that point	Full architecture + learned parameters + weights
When saved	After every N epochs during training	After all training is complete
Purpose	Monitor training progress, resume if interrupted	Final deployment and inference

For example, when training for 10 epochs, checkpoints can be saved after every 2 epochs to check how the model performs on test data at each intermediate stage.

5 Summary

- **GANs** consist of two networks (generator and discriminator) trained adversarially — the generator creates synthetic data from noise, and the discriminator distinguishes real from fake
- **Vanilla GANs** use simple linear (fully connected) layers in both generator and discriminator, suitable for simple tasks like MNIST digit generation
- The generator in Vanilla GAN has four blocks (Linear + BatchNorm + LeakyReLU) taking a latent vector of size 100 and producing a 784-dimensional output (reshaped to 28×28)
- The discriminator has three blocks with Linear + LeakyReLU but **no Batch Normalization**, taking a flattened image (784) and outputting a single real/fake probability
- **DCGANs** replace linear layers with convolutional layers — transposed convolution (kernel_size=5) in the generator and standard convolution (kernel_size=4) in the discriminator
- DCGAN generator produces 32×32 images; real MNIST images are upsampled from 28×28 to 32×32 during training
- DCGAN generator channel progression: $256 \to 64 \to 32 \to 16 \to 1$; discriminator filters: $1 \to 16 \to 32 \to 64$

- **Training order**: Discriminator is updated first, then the generator, because generator loss depends on discriminator output
- **Label convention**: Real = tensor of ones, Fake = tensor of zeros; BCE loss measures deviation from these labels
- **Learning rate** for GANs is typically around 0.0002 — too high causes instability, too low causes slow training
- **Batch normalization** normalizes layer outputs using batch mean and standard deviation
- **Leaky ReLU** prevents dead neurons by providing small non-zero output for negative values, unlike standard ReLU which outputs zero